



ITESO, Universidad Jesuita de Guadalajara

Project #03 – Advanced Trading Strategies Deep Learning

Santiago Reyes Castillo

Camila Daniela Zapata Castañeda

Financial Engineering

Market Microstructures and Trading

October 28th, 2025

Table of Contents

Introduction.....	3
Deep Learning and Trading.....	3
Deep Learning Explanation, Advantages (MLP and CNN) and disadvantages.....	3
Methodology.....	4
Data Preprocessing and Source.....	4
Feature Engineering (Technical Indicators).....	4
Indicators:.....	4
Data Normalization.....	6
Labelling the Dataset	6
Class Balancing.....	7
Data Drift.....	8
Deep Learning Models (Models Explanations)	11
MLFlow (Model Versioning).....	13
Backtesting.....	14
Model Accuracy Metrics	14
Conclusions.....	17
Limitations and Improvement Opportunities.....	17
Sources	19

Introduction

Deep Learning and Trading

The financial markets represent one of the most complex and dynamic environments for analysis and decision-making. They are characterized by their high volatility and random behaviour. Therefore, predicting asset prices has always been considered a challenge for those who want to invest. For this, we resort to technical analysis, mathematical indicators which attempt to decipher patterns in historical data and thus anticipate trends.

However, the effectiveness of these tools is often limited. Indicators can generate false, contradictory, or delayed signals, and may not always capture the impact of the different factors influencing the market. It is for this reason that new tools were sought to combine these indicators with, and this is where our Machine Learning models come in. More specifically, Deep Learning. Unlike other classic statistical models, deep neural networks have the capacity to learn and model extremely complex and non-linear relationships directly from the data.

This project focuses precisely on developing and evaluating a trading system based on Deep Learning models. The main objective is to build a model that uses a set of technical indicators as input to generate buy or sell signals. The goal is to demonstrate that, by processing information from multiple indicators jointly, the Deep Learning model can identify complex correlations and improve prediction accuracy, thereby overcoming the limitations of our previous trading model. This will be accomplished by utilizing neural networks in order to classify market trends, such as a bullish, bearish and neutral trends in a securities' behaviour, to obtain a signal that we can then abide and open positions based on a deep learning model's prediction.

Deep Learning Explanation, Advantages (MLP and CNN) and disadvantages

We utilized two neural network architectures, MLP (Multilayer Perceptron) and CNN (Convolutional Neural Network) to determine which structure is most effective for different aspects of market prediction. The MLP serves as a straightforward baseline for feature-based classification. CNNs, on the other hand, are effective at detecting local temporal patterns in sequences, making them suitable for recognizing short-term trends in the market.

Each architecture has its expected advantages and limitations. MLPs are simple, fast to train, and effective for general feature-based patterns but may struggle with capturing temporal relationships. CNNs efficiently extract localized patterns, improving the detection of short-term trends, but may miss broader market context. By comparing these architectures, we aimed to combine their strengths to enhance the robustness and accuracy of our trading signal predictions, while being mindful of overfitting and the inherent noise in financial markets.

Methodology

The following methodology was developed for this project in order to obtain measurable results, as well as interpretations and correct reproduction of it in other environments. All the steps of the development of the project were built using Git version control, and as will be mentioned further in the report, deep learning models were stored in MLFlow, so scenarios can be reproduced.

Before explaining the process of the project, it is important to briefly go over the functionality of it. The purpose of the model we are going to create is to classify, given the features given by a stock's historical information, in order to generate a buy, sell or hold signal to follow. For this, we obtain data, then features, based on both we label our current data so we can give the model something to follow, in this case these 3 signals will each be a label. Then we can train the model and finally analyze the strategy using a backtesting environment and an statistical approach to find out whether this kind of modelling can be applied to trading. We also implemented a Train, Validation and Test methodology in order to verify the model and strategy's performance.

Data Preprocessing and Source

Firstly, we obtained our data and analyzed it, to see if it was a good option for the project objectives. The data utilized was exported from "FactSet", a large platform for financial information, where you can find as much information about companies and securities, and in this case, historical data for stocks. In this instance, the dataset used consists of the last 15 years of a stock's daily OHLC (Open, High, Low, Close) as well as volume, corresponding to New York Times' market stock. This data ranging from the 10th of October of 2010 to the same day in 2025.

The data from "FactSet" was fairly clean so the only preprocessing utilized was to sort the index of the table so the most recent rows correspond to the end of the table. Then, some information relating to the cumulative and normal returns was erased, as it was easier to calculate by ourselves rather than infer the calculations made by the platform.

All of this was done taking into account and considering that a price behavior is so that it has to be modeled as a time series and not a discontinuous set of information. If not, a lot of information for the model would be leaked or ignored down the training of the model.

Feature Engineering (Technical Indicators)

The next step was to do feature engineering to our data, in order to create enough information for a deep learning model to learn and be trained for. For this, we calculated different technical analysis indicators. These indicators are often used by traders to try and predict a securities trend or risks. There are three type of indicators that were used in this project specifically, and in total, 24 different indicators were used to model the strategy.

Indicators:

Momentum Indicators

Momentum indicators measure the speed and strength of price movements, helping identify trend direction and potential reversals.

- **RSI 7, RSI 14, RSI 21 (Relative Strength Index):** Measures overbought or oversold conditions based on price gains over 7, 14, or 21 periods.
- **Awesome Oscillator:** Calculates momentum by comparing recent moving averages to identify trend changes.
- **KAMA (Kaufman's Adaptive Moving Average):** Adapts to market noise, highlighting the true trend by smoothing price data.
- **ROC (Rate of Change):** Measures percentage change in price between current and n periods ago to detect momentum shifts.
- **Stochastic Oscillator:** Compares closing price to recent price range to indicate overbought or oversold levels.
- **Stochastic RSI:** Applies stochastic calculation to the RSI, increasing sensitivity to momentum changes.
- **TSI (True Strength Index):** Combines momentum and trend by smoothing price changes and their velocity.
- **Ultimate Oscillator:** Combines short-, medium-, and long-term momentum into a single value to avoid false signals.

Volume Indicators

Volume indicators analyze trading volume to confirm trends, identify buying or selling pressure, and signal potential reversals.

- **ADI (Accumulation/Distribution Index):** Assesses if volume supports price moves by comparing close to range.
- **CMF (Chaikin Money Flow):** Measures buying/selling pressure over a period using price and volume.
- **FI (Force Index):** Combines price change and volume to show buying or selling strength.
- **MFI (Money Flow Index):** A volume-weighted RSI reflecting overbought/oversold conditions.
- **NVI (Negative Volume Index):** Tracks price movements on days with lower volume, indicating smart money activity.
- **OBV (On-Balance Volume):** Cumulates volume based on price moves to confirm trends.

Volatility Indicators

Volatility indicators measure the degree of price variation over time, helping assess market risk or uncertainty.

- **ATR (Average True Range):** Averages true price ranges to quantify market volatility.
- **BB High, BB Low, BB Mid, BB Width (Bollinger Bands):** Bands around a moving average expanding/contracting with volatility; width shows volatility degree.
- **Ulcer Index:** Measures downside risk or drawdown length and depth, quantifying price volatility focused on losses.

Data Normalization

For data processing, a feature-scaling strategy specific to each indicator type was implemented. Instead of applying a generic method like StandardScaler to all data, features were grouped according to their intrinsic nature to preserve their predictive power.

The approach was divided into the following groups:

- **Group 1. Oscillators with a Range of [0, 100] or [-100, 100]:** Indicators such as RSI, Stochastic Oscillator, MFI, TSI, and ROC, which operate within a bounded range, were normalized by dividing their values by 100 to scale them to the [0, 1] or [-1, 1] range.
- **Group 2. Price-Based Indicators:** The Kaufman's Adaptive Moving Average (KAMA) and Bollinger Bands (BB) were scaled as a percentage deviation from the asset's closing price. This detrends the price and converts the value into a relative measure indicating how far the indicator is from the current price.
- **Group 3. Volatility Indicators:** The ATR, Bollinger Bands Width, and Ulcer Index, which are on the same scale as the price, were normalized by dividing them by the closing price. This transforms them into a measure of volatility relative to the asset's current price.
- **Group 4. Unbounded Indicators:** Oscillators and volume accumulators like the Awesome Oscillator, ADI, OBV, and Force Index, which lack a clear upper or lower bound, were transformed by calculating their percentage change. This technique helps achieve stationarity, focusing the model on the indicator's rate of change rather than its absolute value, which can grow indefinitely.
- **Group 5. Price and Volume:** The main Price and Volume time series were also converted to their percentage change. This allows the model to be trained on asset returns and volume changes, which are fundamental features in technical analysis and are typically more stationary than raw price levels.

Labelling the Dataset

In order to label the dataset, we set conditions for each label. As mentioned before, the model is going to predict one of three different classifications. A buy signal if there is an upwards trend, a sell signal with a downward trend and a hold signal if there is no sufficient evidence for a trend.

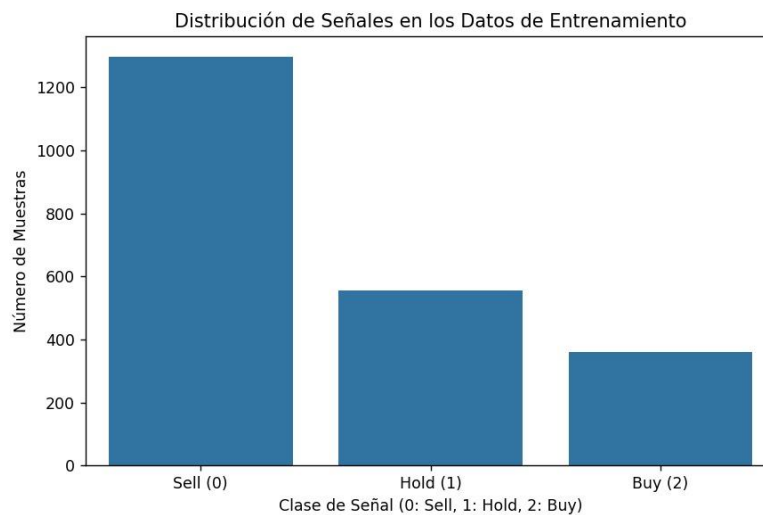
For this, the label was created with a lookback period, where, if the price given a set window of shifts (lookback period) in the future, was greater than the price in the past day, plus or less an “alpha” coefficient, we would take said day as a buy or sell signal. In other words, in a given day, if after a set number of periods, the price was higher than that price plus a coefficient, a buy signal is given, and vice versa, if it was lower, a sell signal would be set, if not, a hold signal was to be put as a target. This way, we can also monitor how strong a trend has to be, and in how long it has to be “seen” in order for the model to interpret as an opening for a position.

The “alpha” coefficient used in this project was 5%. This way, the model receives a similar quantity of buy and sell signals, and that way, we can be more precise in its training. This is because naturally, the market is hard to read, trends are never predicted with full precision, so using a higher coefficient would result in a majority of hold signals that would make the strategy extremely passive and not operate for many given opportunities.

However, this also sets up another issue, that being class balancing in classification problems. Even with a low coefficient for the labelling, most of the features produce hold signals, so it is important to balance classes accordingly, for the model not to be trained with only one majority of classes.

Class Balancing

During the project, we noticed that the imbalance in the dataset was really notorious, before adjusting the labelling coefficient, we had 80% of the features predicting a hold signal, which greatly affected our models training, being that most of the predictions were for the signal to hold, but that was because most of the dataset said so as well, so it would not be correctly prepared to predict other trends.



In trading data, “hold” signals are naturally far more frequent than “buy” or “sell” signals. This phenomenon, known as class imbalance, is problematic because a model could achieve high accuracy by simply always predicting the majority class, rendering it useless for actual trading. To mitigate this issue, a class weighting technique was implemented during the model's training

phase. However, by modifying the way the dataset was labelled, signals are more sensible and not every behaviour is modelled as a hold suggestion.

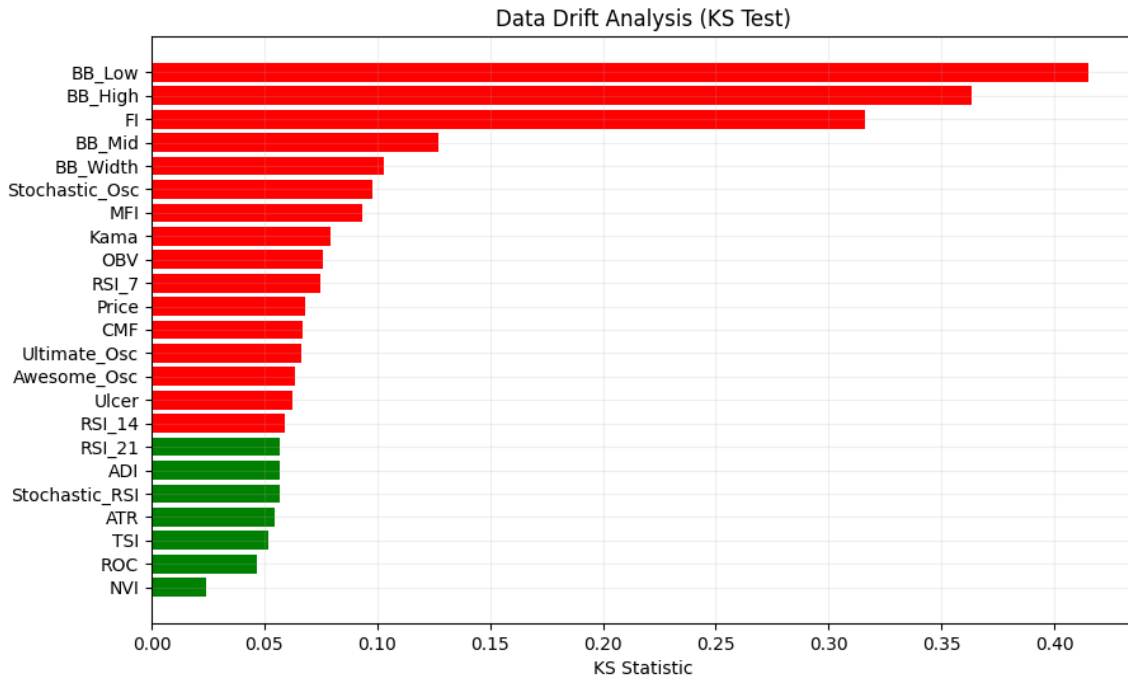
The process for balancing is as follows:

1. Using scikit-learn's `compute_class_weight` function with the `class_weight='balanced'` argument, weights were calculated for each of the three classes (0, 1, and 2) based on their frequency in the training set (`y_train`).
2. The function automatically assigns a higher weight to minority classes (buy and sell) and a lower weight to the majority class (hold). The weight is inversely proportional to the class frequency.
3. These weights are passed to the model during training. As a result, the model's loss function more heavily penalizes misclassifications of "buy" and "sell" classes. This forces the model to pay closer attention to these scarcer but more critical signals, improving its ability to identify them correctly.

Data Drift

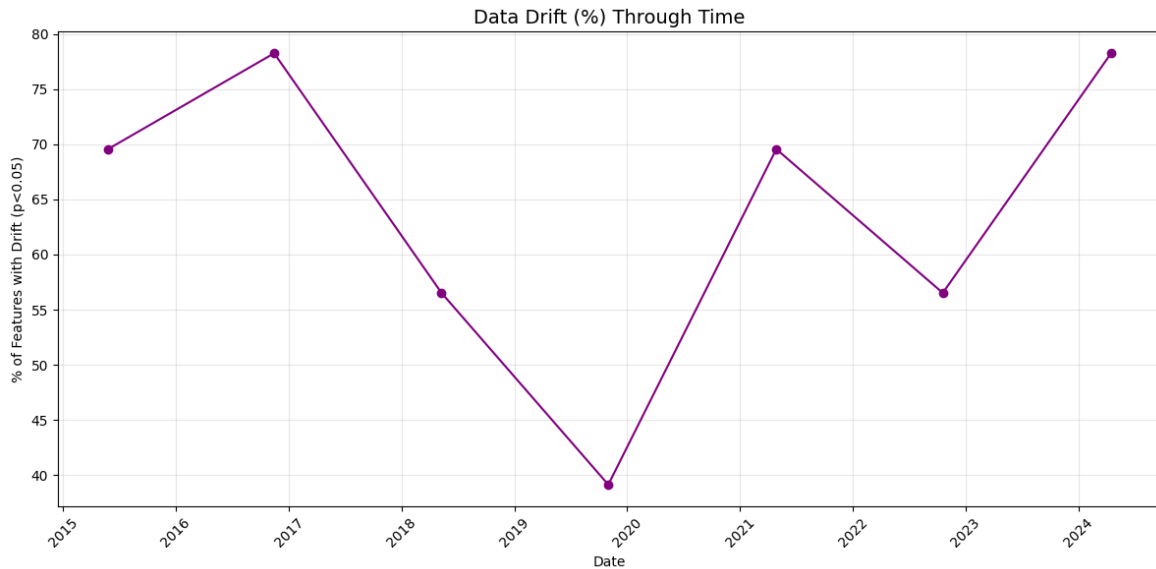
As mentioned earlier, an asset's price behaves like a time series, and this means that the data provided by it will very probably be affected by that condition. In this instance, the data used for the model follows a certain distribution, that may or may not change as time goes on. To analyze this aspect of the dataset, a data-drift analysis was made.

Firstly, we wanted to know if the distributions for data in the train and test periods were statistically different, and for this, we tested utilizing a 2 sample Kolmogorov-Smirnov test, that will provide a statistic and p-value, that given a 95% confidence interval, will tell us if any given feature has or not, a statistically significant difference in it's distribution during the first and last period of analysis (train against test)



We can see in the graph above, that out of 24 different indicators, only 7 of them keep their distribution the same (marked with a green bar above) while the rest of them do not, and some of them show significant evidence of being statistically different to the data that trained the model. This means that more often than not, the predictions made by the model may be flawed, since it gets used to a certain distribution in the training period, but against a testing scenario, and different distributions, the data received varies heavily, implying it might no longer be useful.

Even then, it is important to analyse how this happens through time, in order to find an adequate solution for this drift in data. Below is a graph that shows the percentage of features from the dataset that provide a p-value higher than 5% (out of the confidence interval) and so it is possible to analyse in which period features' distribution changes the most. The baseline sample that was used to analyse the differences is the first third of the dataset, corresponding to around 3 years of information.



The graph above shows the percentage of features whom indicator is below the confidence level, at different moments of time. For this statistical test, a low p-value represents that there is enough evidence to call a difference in distribution, so, a high percentage of features below that threshold means that there is a significant difference in distribution for a lot of features.



If we see the plot for the price history (shown above) we can see that the data starts to stabilize when the price changes trend, from a bullish trend to a more horizontal trend. This suggest that this model will probably be better at detecting hold signals, and not necessarily buy or sell operations. Since it was trained with distributions more similar to this market condition

The correct solution for this problem would be to design a script that dynamically adjusts the model's training, so if in any point at time during the monitoring of the model, the data start drifting enough for the model to deviate from correctly predicting, to train again and find new parameters that adjust better to the new information that goes into it. However, this solution is

ahead of this project's range, so it will be analyzed as a potential solution but will be overlooked for this specific project.

Deep Learning Models (Models Explanations)

MLP: Architecture diagram, layer descriptions, parameters

This model, also known as a Dense Neural Network (DNN), processes time-series data after flattening it, treating each lookback sequence as a single feature vector.

Layer Descriptions

The model's architecture is defined in the `create_dnn_model` function and consists of the following layers:

1. **Flatten Layer:** This is the first layer, and its function is to transform the input data from a shape of `(lookback_period, num_features)` into a one-dimensional vector.
2. **Dense Layer (1):** The first hidden layer, with 128 neurons and a ReLU (Rectified Linear Unit) activation function. This layer learns complex non-linear relationships between the input features.
3. **Dropout Layer (1):** Applies regularization to prevent overfitting. During training, it randomly "turns off" 30% of the neurons from the previous layer, forcing the network to learn more robust patterns.
4. **Dense Layer (2):** A second hidden layer with 64 neurons and ReLU activation. It reduces dimensionality and continues to extract higher-level features.
5. **Dropout Layer (2):** Another 30% dropout layer for continued regularization.
6. **Dense Layer (3):** The third hidden layer with 32 neurons and ReLU activation.
7. **Output Layer:** The final layer. It is a Dense layer with 3 neurons, one for each signal class (0: sell, 1: hold, 2: buy). It uses the Softmax activation function, which converts the network's outputs into a probability vector where the sum of the probabilities for the three classes equals 1.

Parameters

The number of trainable parameters in a Dense layer is calculated as $(\text{inputs} * \text{neurons}) + \text{neurons}$. Dropout layers have no parameters. The model's total parameters are the sum of the parameters from all its Dense layers.

CNN: Convolutional structure, kernel sizes, pooling strategy

This model is designed to identify local patterns in data sequences. The Conv1D layer acts as a feature detector that slides across the time series.

Convolutional Structure

Defined in `create_cnn_model`, the structure is as follows:

1. **Conv1D Layer:** The main layer. It applies 64 filters (or "kernels") to the input sequence. Each filter learns to detect a specific pattern (e.g., a rapid rise in an indicator). It uses ReLU activation.
2. **MaxPooling1D Layer:** Its purpose is to reduce the dimensionality of the output (downsampling) while retaining the most important information.
3. **Dropout Layer:** A 30% dropout is applied for regularization.
4. **Flatten Layer:** Flattens the output of the previous layers to connect it to dense layers.
5. **Dense Layer:** A hidden layer with 50 neurons and ReLU activation to perform classification based on the extracted patterns.
6. **Dropout Layer:** Another 30% dropout.
7. **Output Layer:** Identical to the MLP's, with 3 neurons and Softmax activation to produce the probabilities for each signal.

Kernel Size

The kernel size is set to `kernel_size=3`. This means each of the 64 filters analyzes the data sequence in windows of 3 time steps, which is ideal for capturing short-term patterns.

Pooling Strategy

MaxPooling1D is used with a `pool_size=1`. Typically, MaxPooling reduces the sequence size. However, with `pool_size=1`, the layer performs no dimensional reduction. In practice, with this configuration, this layer has no significant effect on the data's structure.

Training specifications: loss function, optimizer, batch size, epochs

- **Loss Function:** `sparse_categorical_crossentropy` is used. This is the standard choice for multi-class classification problems where the true labels (`y_train`) are integers (0, 1, 2) rather than one-hot vectors. It measures how well the model's predicted probabilities match the actual class.
- **Optimizer:** Adam is used. It is a popular and efficient optimization algorithm that adaptively adjusts the model's learning rate and generally performs well with minimal configuration.
- **Batch Size:** Set to 10 (`batch_size=10`). The model processes the training data in batches of 10 sequences at a time, updating its internal weights after each batch.
- **Epochs:** The maximum number of epochs is set to 100. However, EarlyStopping is implemented with `patience=10`, which automatically stops training if the validation loss (`val_loss`) does not improve for 10 consecutive epochs, restoring the best-found weights.

Class weighting formula and application

The code uses scikit-learn's `compute_class_weight` method with the `class_weight='balanced'` parameter. This mode automatically calculates the weights for each class using the following formula:

$$w_j = \frac{n_{samples}}{n_{classes} * n_{samples_j}}$$

Where:

- w_j is the weight for class j .
- $n_{samples}$ is the total number of samples in the training data.
- $n_{classes}$ is the total number of classes (3 in this case).
- $n_{samples_j}$ is the number of samples belonging to class j .

Application: The resulting weight dictionary (`class_weights_dict`) is passed to the `model.fit()` method via the `class_weight` argument. In practice, this tells the model to pay more attention to errors made on minority classes. When calculating the loss function, the error of an example from a minority class is multiplied by its weight, amplifying its impact and forcing the model to learn to identify those classes better.

MLFlow (Model Versioning)

The MLflow platform was implemented for experiment management and tracking, ensuring the reproducibility of each training run and maintaining an organized record of all models, their parameters, and their results. The logging structure is located in the `mlflow_manager.py` script. The main function, `run_mlflow_experiment`, encapsulates the entire experiment lifecycle as follows:

- **Starting a "Run":** Each time a model is trained, a new "run" is initiated in MLflow using `mlflow.start_run()`. Each run is assigned a name (in this case, the model type, like 'DNN' or 'CNN').
- **Logging Parameters:** At the start of each run, all hyperparameters used for that specific training (such as learning rate, number of neurons, batch size, etc.) are logged via `mlflow.log_params(params)`.
- **Logging Metrics:** Upon completion of training, key performance metrics on the test set, such as loss (`final_test_loss`) and accuracy (`final_test_accuracy`), are logged with `mlflow.log_metric()`.
- **Storing the Model (Artifacts):** The trained model object is saved as an artifact within the MLflow run using `mlflow.tensorflow.log_model()`. This allows us to not only review its performance but also to load and use it later for inference or further analysis.

Backtesting

After obtaining our models, and being able to predict with them in independent datasets, we test these predictions in a simulated-real environment for trading, in which we start with a given quantity of cash, and using exclusively the signals that the model predicted, open operations in the direction given (long or short) and if the prediction suggests to hold, we do so. Operations will be closed whenever they hit either a Stop-Loss or Take-Profit threshold. Additionally, the number of shares bought or sold per operation will stay constant throughout the testing, and these parameters will be arbitrarily chosen.

For this to work appropriately, we have to take into account costs that happen in real life scenarios. Mainly commission costs as well as a borrow rate for any given short position that we open and maintain for longer than one period.

Commission costs apply when we open and close any position, and are calculated as a percentage to be paid, given the present price of the stock. For this environment, we considered commissions as a 12.5% constant cost. The borrow rate, as mentioned above, is the cost of having a short position opened, and it is paid every period that these types of positions stay open. Similarly, we considered it as a constant annual 0.25%. This way, we can open positions, and pay the costs of doing so in a similar manner to the real market.

In the same fashion we set a Stop-Loss and Take-Profit of 10% for both, so we cash in big or lose big as well, and trust in our model's accuracy. The number of shares was set to 1200, because we set the starting cash of the strategy to be one million dollars, and the stock chosen is cheap, with a minimum price of around 5 dollars and a maximum of 50. The project started with \$10,000 dollars to operate with.

The main takeaway from the backtesting is to know whether the trading strategy would payoff or not, and how much it could produce as well as cost given the conditions mentioned earlier.

Even if the conditions for the backtesting were strict, there are still some big limitations to be taken into account. Firstly, we did not take into account margin accounts. These are often a very important aspect of short positions specifically and subtract big amounts of money to the cash at hand of a given portfolio. Similarly, not implementing spread costs, and simulating a frictionless environment means that there are some hidden costs that in the real market would negatively affect the value of a portfolio. At the same time, this strategy consists of only a few models for only one stock, which means the portfolio is not diversified enough to mitigate the risks that come along with the market. These limitations must be considered when analyzing the results of said strategy, to not conclude from erroneous information.

Results and Analysis

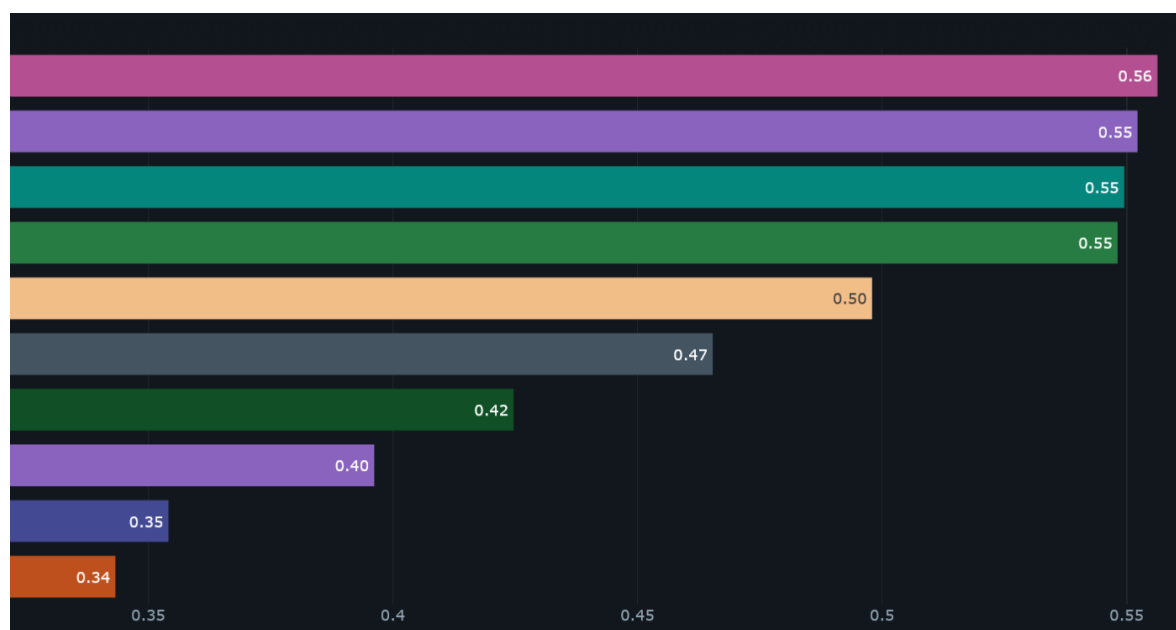
Model Accuracy Metrics

Below, a table for comparison between the resulting metrics for both models in their test period. We can see that the Convolutional Neural Network has better metrics overall. Annualized return is at 2% compared to 0.82% of the Dense Neural Network. It has a better overall performance

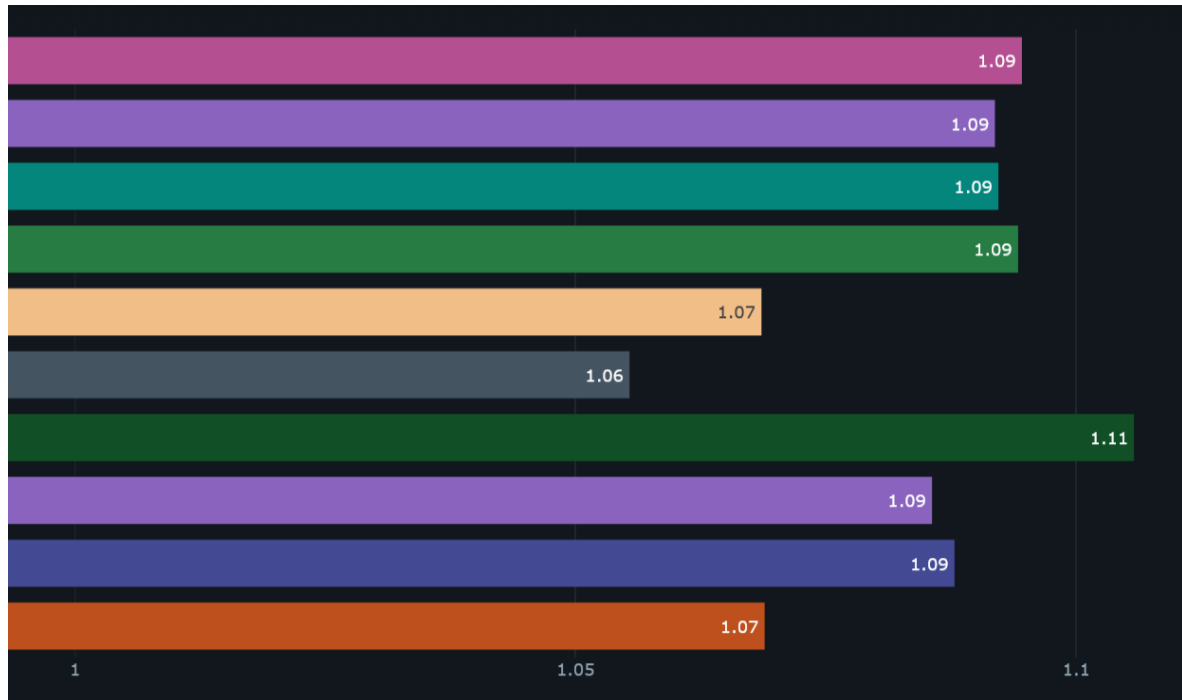
as well, with a higher Calmar and Sortino ratio. Nevertheless, both model's win rate is similar to each other.

The most important finding is that there is probably a correlation between the model's accuracy metric and its performance overall, as the highest accuracy obtained with all the models, which belongs to the CNN listed below, is also the highest return obtained. This means that aiming for better performance metrics will also add to the performance of the strategy.

Metric	DNN	CNN
Annualized Return	0.82%	2.00%
Annualized Volatility	4.57%	4.40%
Max Drawdown	2.975%	4.268%
Calmar Ratio	0.27605	0.46957
Sortino Ratio	0.26803	0.65358
Win Rate	48.10%	46.34%
Accuracy	0.50	0.56
Loss	1.07	1.09

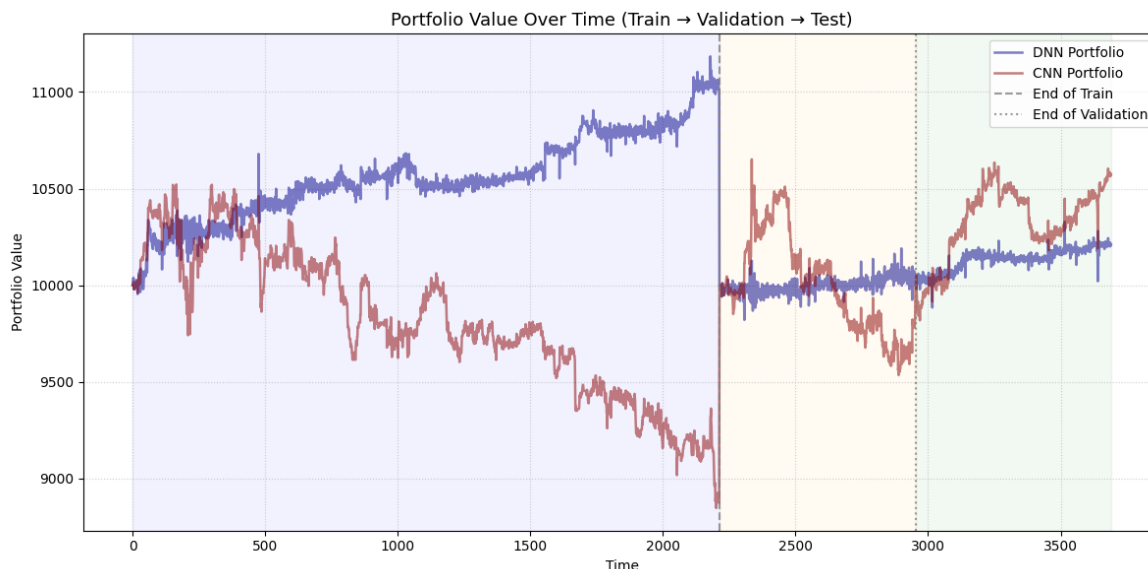


Final accuracy for the testing (10 best models)



Final Loss Value for the testing (10 best models)

Conclusions



Talking about the strategy's performance, we can see that during training, both models go to different directions. The convolutional network can't recognize the trend as quickly as the dense neural network. However, as time goes on, during validation and test, it is more clear on its decisions. Nevertheless, the dense network seems to be way more precise and less volatile than the convolutional. This means that maybe, an investor that is more adamant to risk may choose to work with a more complex model, and someone that is not that interested in risky decisions may choose another model to base his strategy on.

It is interesting to see that the change in value is not that high. This can be traced back to the backtesting strategy parameters, maybe by increasing the number of shares traded per operation, we could potentially see more pays but more losses at the same time. Similarly, playing around with the thresholds to open and close operations might help either stabilizing or maximizing returns and losses.

Limitations and Improvement Opportunities

There are some limitations for this project, that if addressed, could better represent what a more complex strategy could accomplish. Regarding the backtesting strategy and environment, as mentioned during the backtesting section, not every cost is considered, and there is also friction and spread costs that could be modelled as well.

The area where there are more opportunities, is definitely the parameters of the model and strategy. It is possible to optimize parameters and hyperparameters given conditions given by the model. But at the same time the model can also be improved. For example, readjusting the model's training every few weeks, or whatever time window suggested would greatly improve performance and metrics. It was seen with the data-drift analysis that given the architecture of the dataset as a time series, parameters cannot stay constant throughout the model, they must be readjusted.

Overall, this project was a good experiment to try and adapt a trading strategy to a Machine Learning workflow. Utilizing MLFlow to keep track of the different models helps us be able to reproduce scenarios and adds a layer of practicality to the project. It is important to also understand the models, and learn to

Sources

1. RSI (Relative Strength Index) - Alpha Vantage:
https://www.alphavantage.co/relative_strength_index_rsi/
2. Awesome Oscillator - Upstox:
<https://upstox.com/learning-center/share-market/mastering-the-awesome-oscillator-a-comprehensive-guide-for-traders/>
3. Kaufman Adaptive Moving Average (KAMA) - TrendSpider:
<https://trendspider.com/learning-center/what-is-the-kaufman-adaptive-moving-average/>
4. Rate of Change (ROC) - Angel One:
<https://www.angelone.in/knowledge-center/online-share-trading/roc>
5. Stochastic Oscillator - GoodCrypto:
<https://goodcrypto.app/the-ultimate-guide-to-trading-a-stochastic-oscillator-stoch/>
6. Stochastic RSI - TrendSpider:
<https://trendspider.com/learning-center/stochastic-rsi-understanding-the-basics/>
7. True Strength Index (TSI) - TrendSpider:
<https://trendspider.com/learning-center/true-strength-index/>
8. Ultimate Oscillator - TrendSpider:
<https://trendspider.com/learning-center/the-ultimate-oscillator-understanding-and-using-it-for-trading/>
9. RSI (Relative Strength Index) - Fidelity Investments:
<https://www.fidelity.com/learning-center/trading-investing/technical-analysis/technical-indicator-guide/RSI>
10. Awesome Oscillator - Alchemy Markets:
<https://alchemymarkets.com/education/indicators/awesome-oscillator/>